

Лабораторная работа № 2 · Вариант В

Интегрирование методом средних прямоугольников

Реализованы последовательная и параллельная программы на C для вычисления $I = \int_0^1 \cos(x^2) dx$ методом средних прямоугольников; OpenMP использует **parallel for reduction(+:sum)** и **schedule(static)**. Программа позволяет задавать пределы, число разбиений и потоков, выводит интеграл и время; дополнительно реализовано удвоение числа разбиений до достижения заданной оценки точности.

Условия и методика: Apple M4, 10 ядер (4 производительных + 6 энергоэффективных), macOS 26.3.1; Apple Clang 17.0.0, libomp 23.1.2, -O2, C11, без fast-math. Время через `omp_get_wtime()`: медиана 3 отдельных запусков, порядок конфигураций перемешан; вывод не учитывается, создание потоков и редукция учитываются, `OMP_DYNAMIC=FALSE`. Ускорение $S = T_{seq} / T_{omp}$.

1. Влияние числа потоков: $n = 10^8$

Последовательная версия: $T_{seq} = 0,176231$ с.

Потоки	Время OpenMP, с	Ускорение
1	0,185876	0,95
2	0,099688	1,77
4	0,059052	2,98
8	0,043866	4,02
16	0,037242	4,73

2. Влияние числа разбиений: 8 потоков

n	Значение интеграла	Время, с	Абс. ошибка
10^4	0,9045242386014976	0,0001321	$7,01 \times 10^{-10}$
10^5	0,9045242379072851	0,0001760	$7,01 \times 10^{-12}$
10^6	0,9045242379003374	0,0006242	$6,53 \times 10^{-14}$
10^7	0,9045242379002679	0,0050540	$4,18 \times 10^{-15}$
10^8	0,9045242379002706	0,0438659	$1,48 \times 10^{-15}$

Эталон $I \approx 0,90452423790027208147$ вычислен независимо по ряду $\sum_{k=0}^{\infty} (-1)^k / ((2k)! (4k + 1))$ (70 слагаемых, десятичная арифметика с точностью 70 цифр). Значения в таблице - медианы 3 запусков.

3. Вычисление с заданной точностью

В PDF не указаны ϵ и условие остановки, поэтому принято $\epsilon = 10^{-8}$, $n_0 = 1$, оценка Рунге $R = |I_{2n} - I_n| / 3 \leq \epsilon$. **$n = 4096$** - первое подходящее число в последовательности удвоений: для 2048 оценка $R \approx 1,67 \times 10^{-8}$, для 4096 $R \approx 4,18 \times 10^{-9}$. Обе версии дали $I \approx 0,904524242079904$; истинная ошибка $\approx 4,18 \times 10^{-9}$. Время всего уточнения: **0,000016928 с** последовательно и **0,000757933 с** на 8 потоках.

Вывод

При $n = 10^8$ увеличение числа потоков с 1 до 16 уменьшило время OpenMP с 0,185876 до 0,037242 с. Ускорение относительно последовательной версии составило 4,02 раза на 8 потоках и 4,73 раза на 16 потоках. После 8 потоков прирост замедлился, что согласуется с ограниченным числом и неодинаковой производительностью ядер. При увеличении n с 10^4 до 10^5 ошибка уменьшилась примерно в 100 раз, как ожидается для метода второго порядка. При дальнейшем росте n влияние округления double становится существенным, поэтому точность не обязана улучшаться пропорционально n^2 . Reduction устраняет гонку данных: потоки независимо накапливают локальные суммы и объединяют их после цикла. Это позволяет избежать синхронизации при каждом прибавлении, необходимой при использовании `atomic` или `critical`. Для короткого адаптивного расчёта последовательная версия быстрее, поскольку расходы на запуск потоков и повторные редукции превышают выигрыш от параллелизма.